

BURP

Paloma Casabona
Miguel Hernández

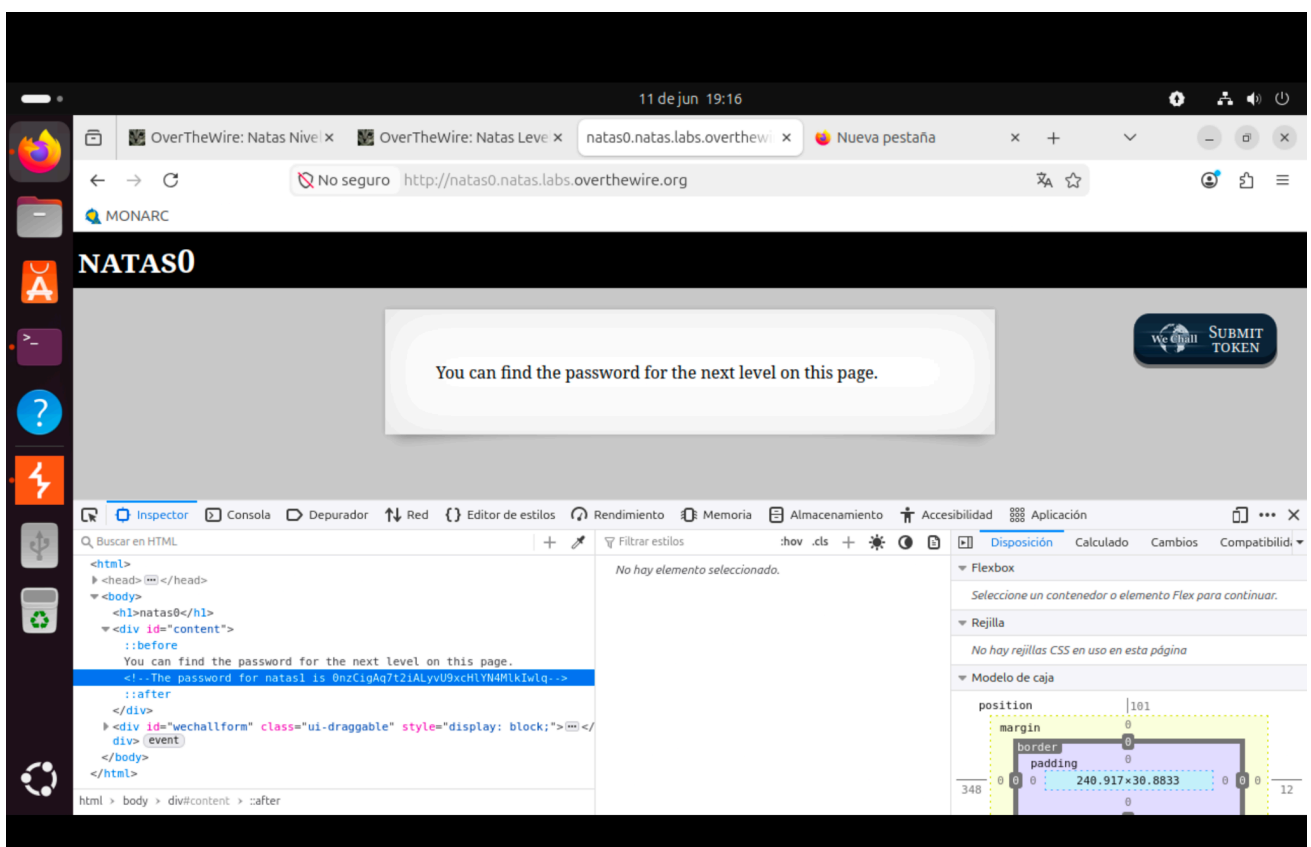
ÍNDICE

TAREA BURP

Utilizar burp para llegar lo mas lejos posible en:

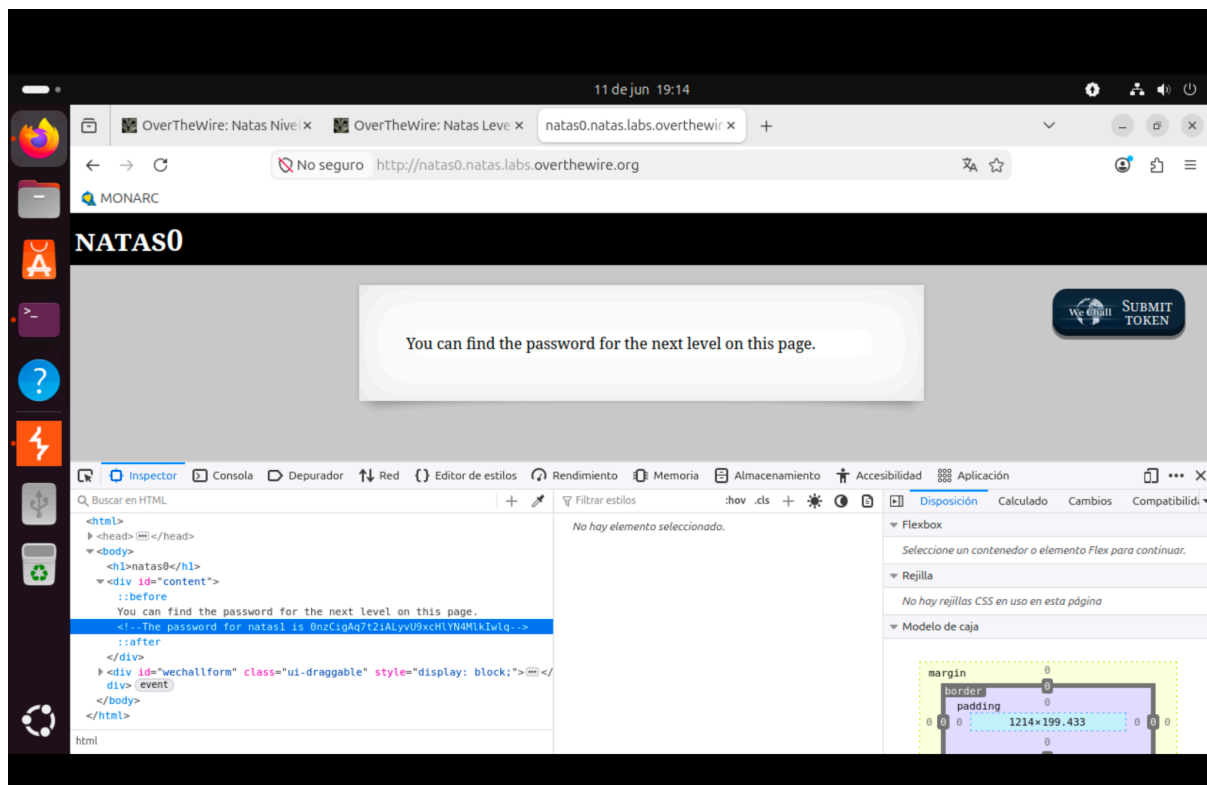
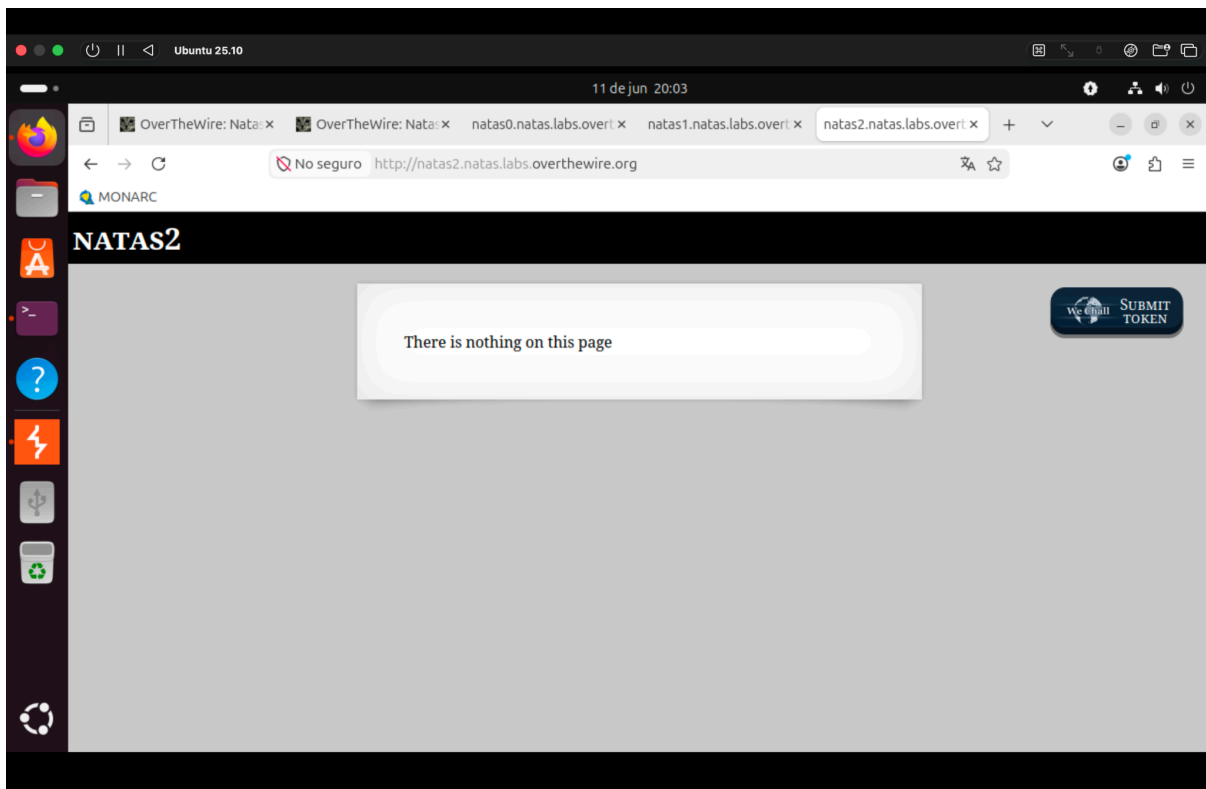
<https://overthewire.org/wargames/natas/natas0.html>

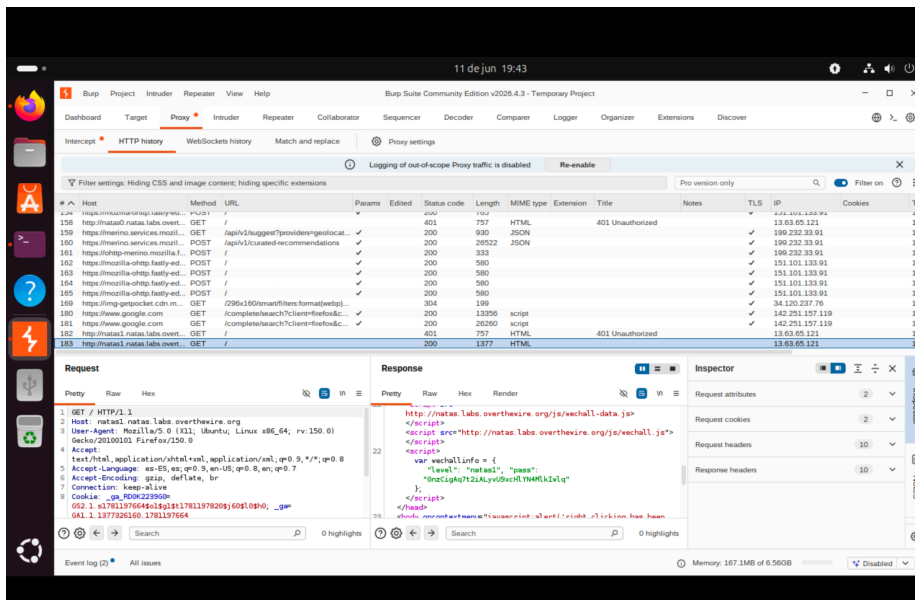
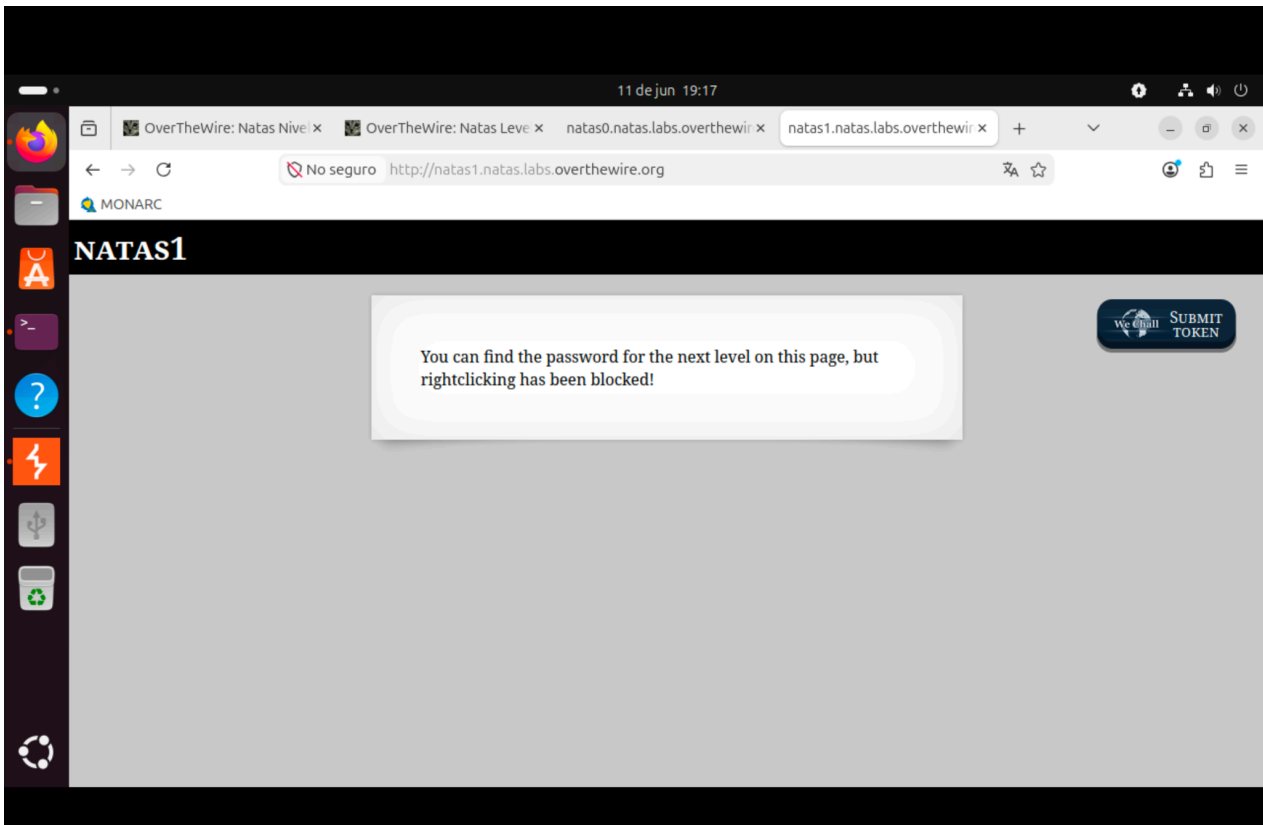
- Realizar los ejercicios de Juiceshop
- Zero Stars
- Expired coupon
- Forged Review
- Payback Time

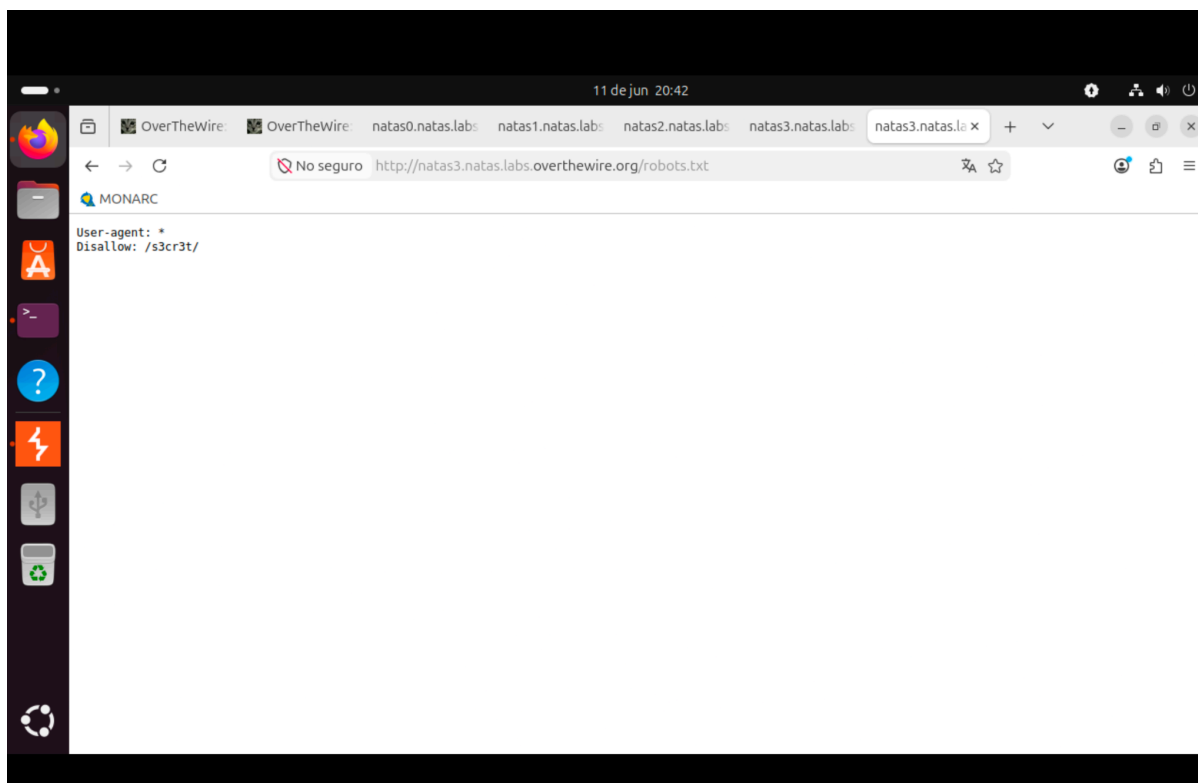
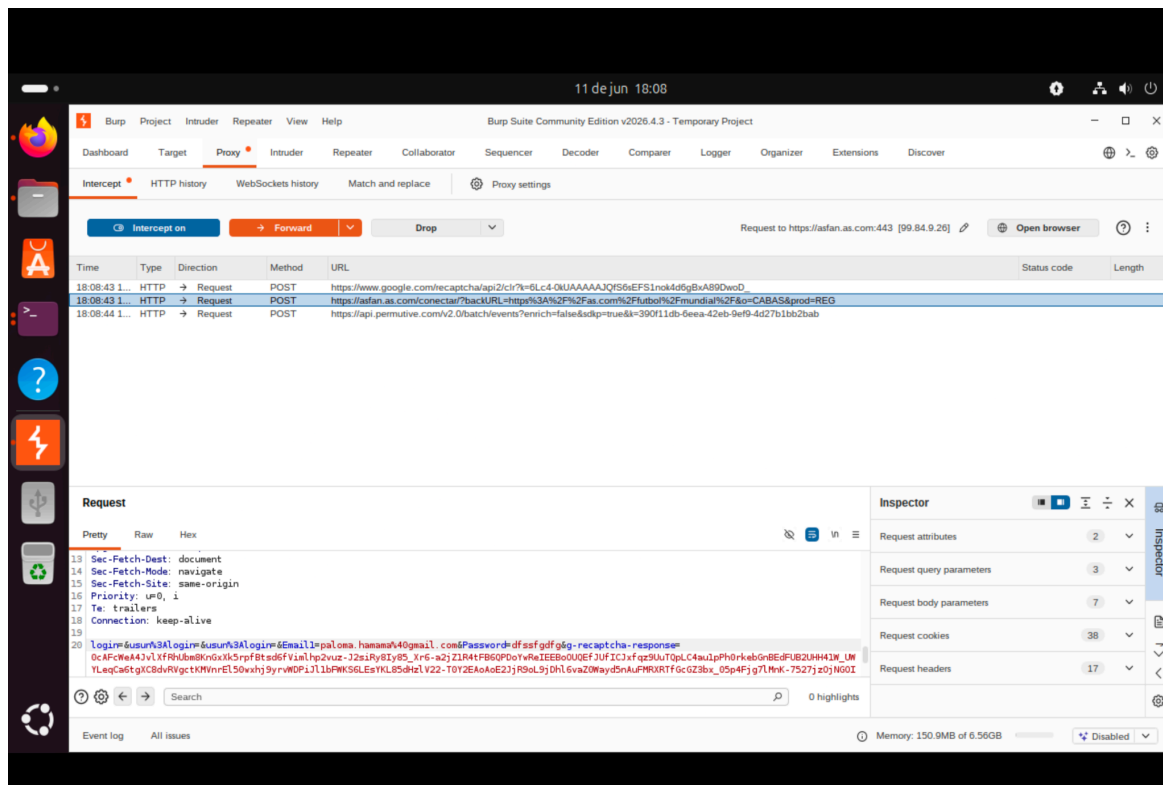


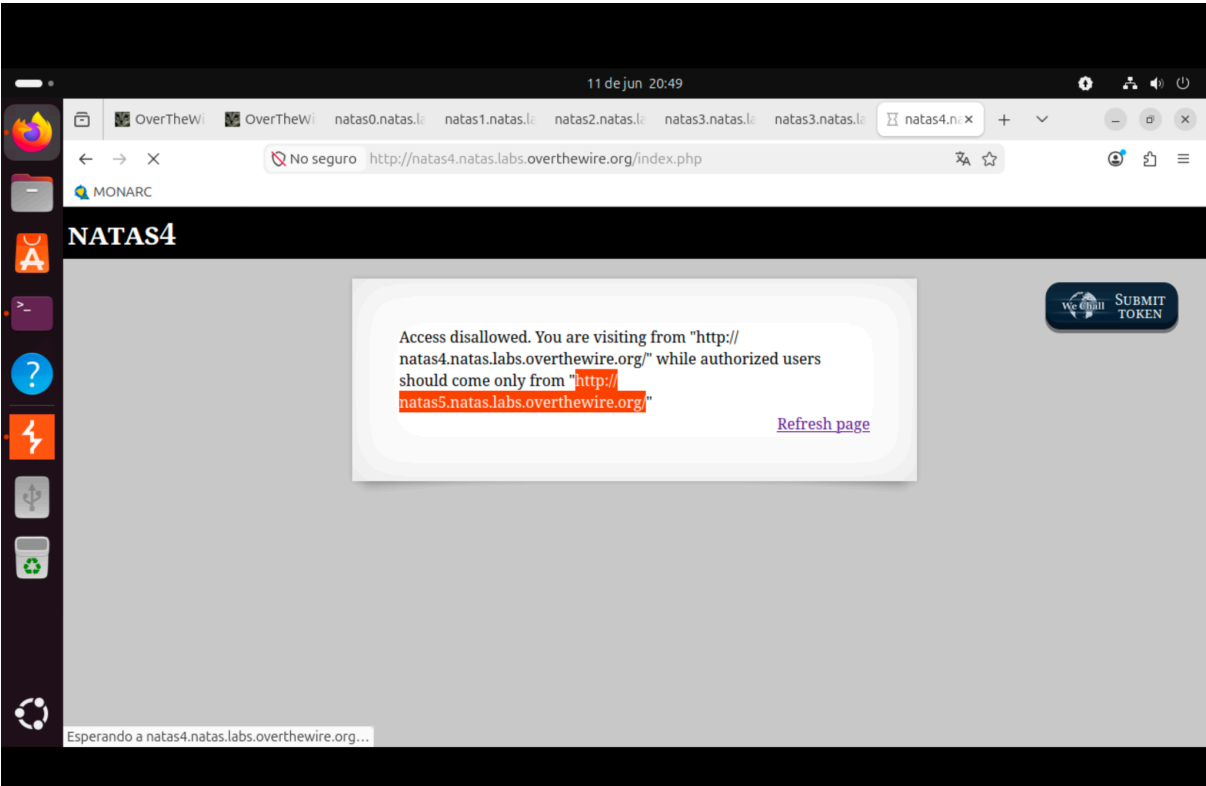
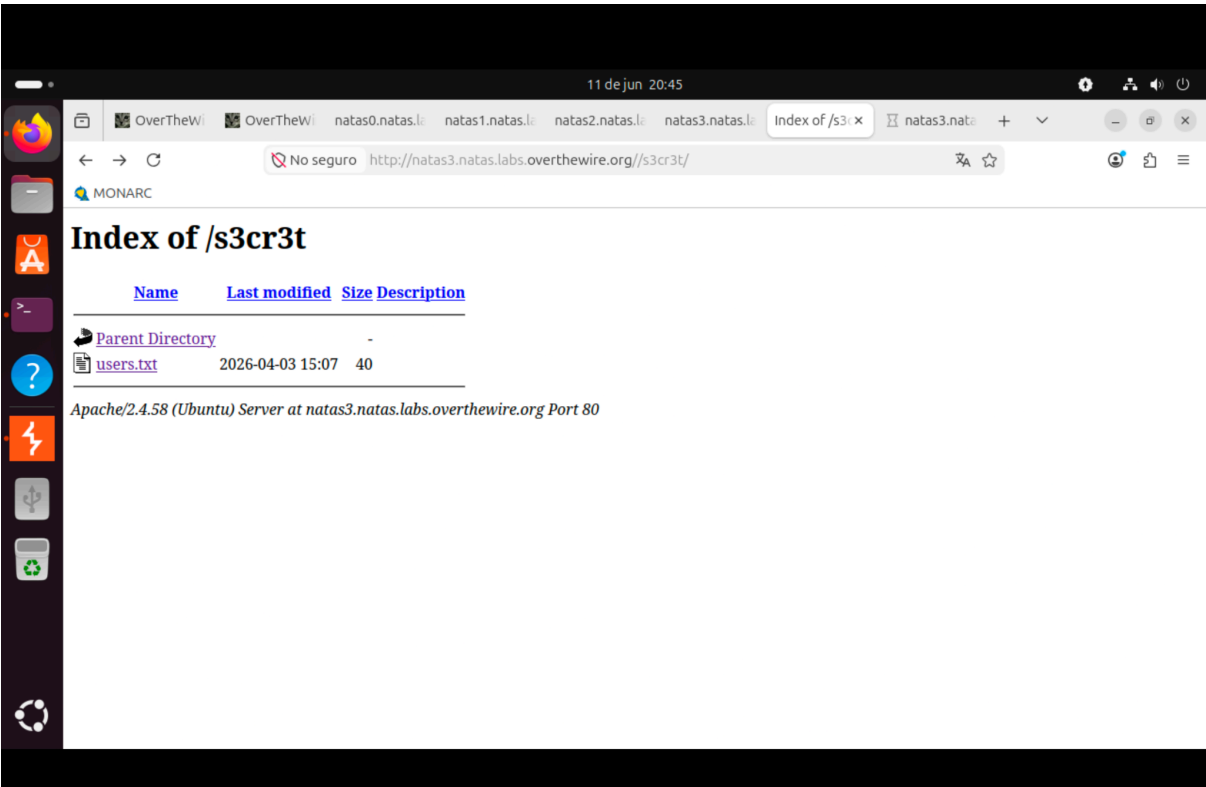
The screenshot shows the Burp Suite interface with the 'Target' tab selected. The Site map on the left lists various domains, including '1x1.a-mo.net', 'prebid.a-mo.net', 'sync.a-mo.net', 'lb.adnms.com', 'dcs.adobedc.net', 'edge.adobedc.net', 'assets.adobedtm.com', 'direct.adsrv.org', 'aax-eu.amazon-adsystem.com', 'c.amazon-adsystem.com', 'web-banner.ads.aps.amazon-adsystem.com', 'as.com', 'cp.as.com', 'dd.as.com', 'static.as.com', 'stories.as.com', 'hibb.casalemedia.com', 'grid-bidder.criteo.com', 'diariosa.disqus.com', 'tritondigital-match.dotomi.com', 'securepubads.g.doubleclick.net', 'fundingchoicesmessages.google.com', 'www.google.com', 'db38269d4f3b9f53849a96a9c04ac2e.safef', and 'nasaad7.norolasdfration.com'. The main panel displays the HTTP history for 'as.com', showing a GET request to 'https://as.com' with a status code of 200. The response is an HTML document titled 'AS.com - Diario online d...'. The Inspector panel on the right shows the request details, including the 'Cookie' and 'User-Agent' headers.

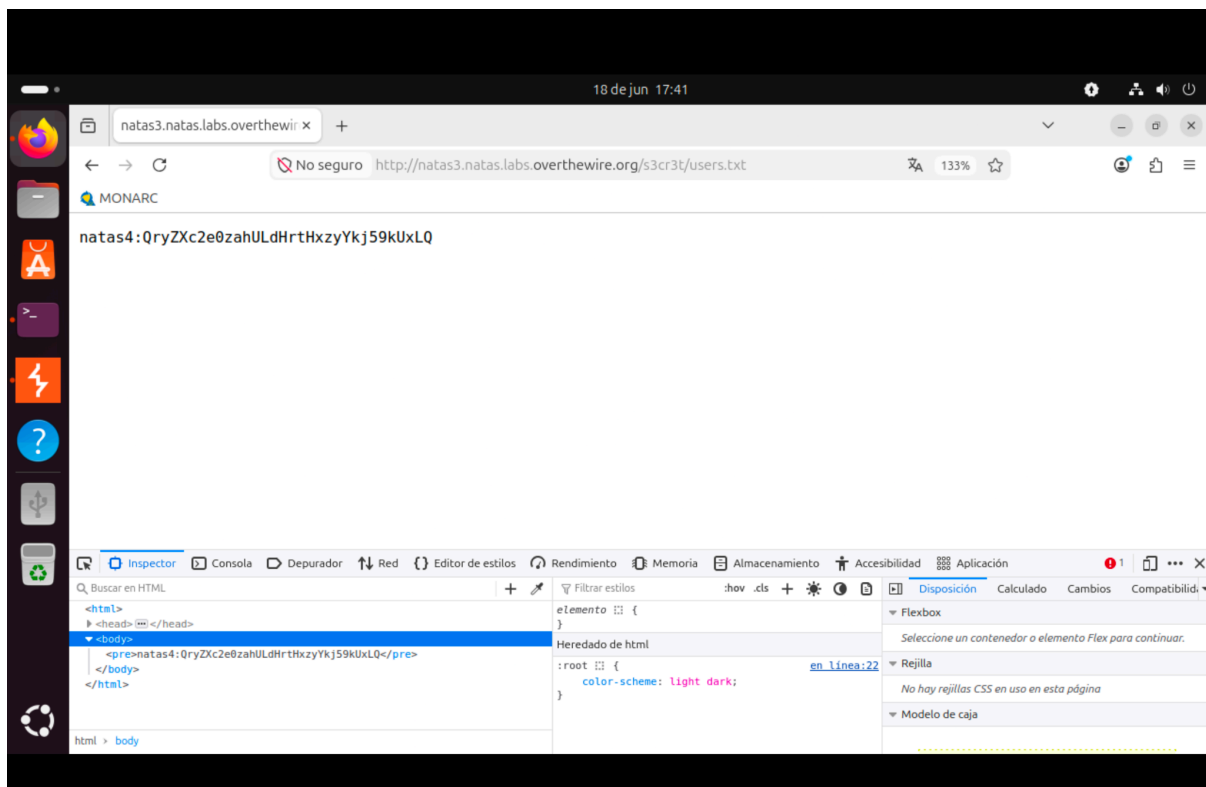
The screenshot shows the Burp Suite interface with the 'Proxy' tab selected. The HTTP history table lists several requests, including a POST request to 'https://web-messaging.insurads.com/v2.0/internal/metrics' and a POST request to 'https://events.newsroom.bi/ingest.php'. The 'Request' panel on the left shows the details of the selected request, including the 'Host' header 'as.com' and the 'User-Agent' header 'Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7; AppleWebKit/537.36; Chrome/104.0.0.0 Safari/537.36)'. The 'Inspector' panel on the right shows the request details, including the 'Host' header 'as.com' and the 'User-Agent' header 'Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7; AppleWebKit/537.36; Chrome/104.0.0.0 Safari/537.36)'. The Event log at the bottom shows the sequence of events, including the request and response.











Hallazgo: Exposición de directorios sensibles mediante robots.txt

URL:

<http://natas3.natas.labs.overthewire.org/robots.txt>

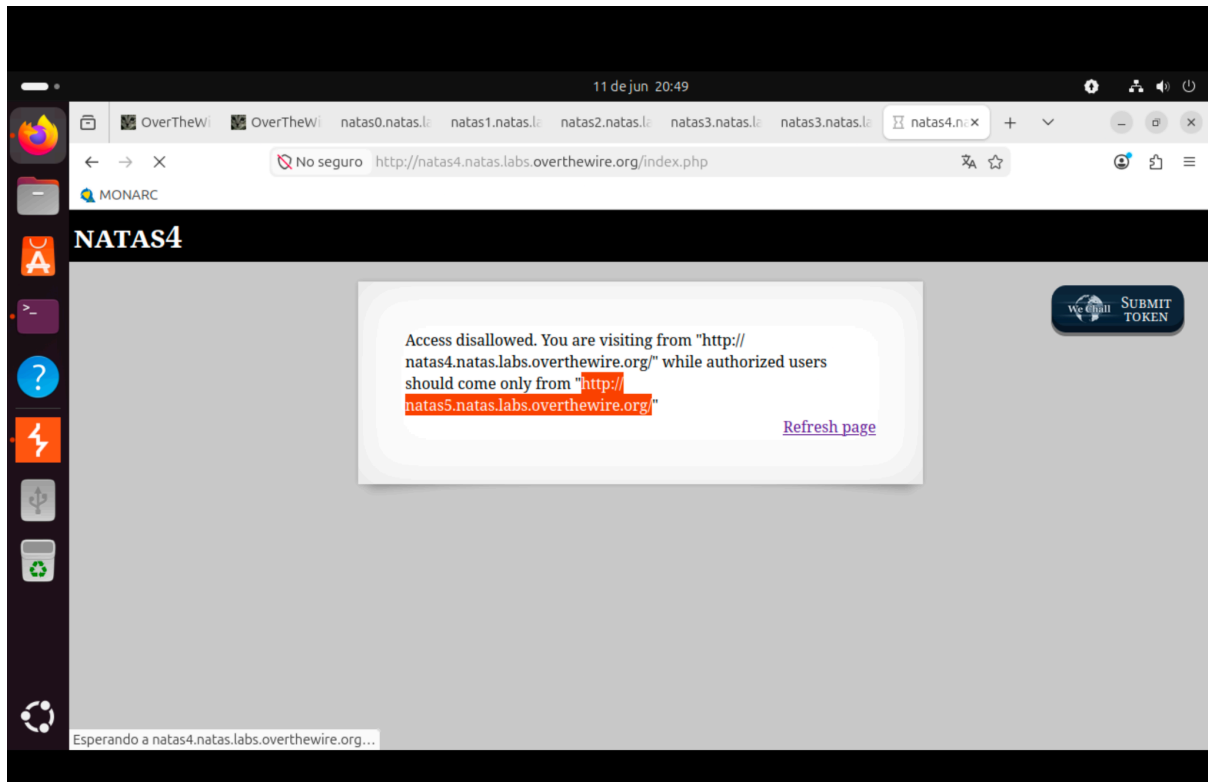
Tipo:

Divulgación de información a través del fichero robots.txt

Impacto:

El fichero `robots.txt` revela la existencia del directorio `/s3cr3t/`, que contiene información que el administrador pretende ocultar a los motores de búsqueda. Un atacante puede utilizar esta información para identificar recursos sensibles y continuar con el proceso de enumeración de la aplicación.

Es importante destacar que `robots.txt` no constituye un mecanismo de control de acceso, sino una recomendación dirigida a los motores de búsqueda. Cualquier usuario puede acceder al contenido del fichero y descubrir rutas internas potencialmente sensibles.



Cómo se descubrió:

Durante la fase de reconocimiento manual se accedió al fichero `robots.txt` del sitio web.

Comando utilizado:

```
curl http://natas3.natas.labs.overthewire.org/robots.txt
```

Resultado obtenido:

User-agent: *

Disallow: /s3cr3t/

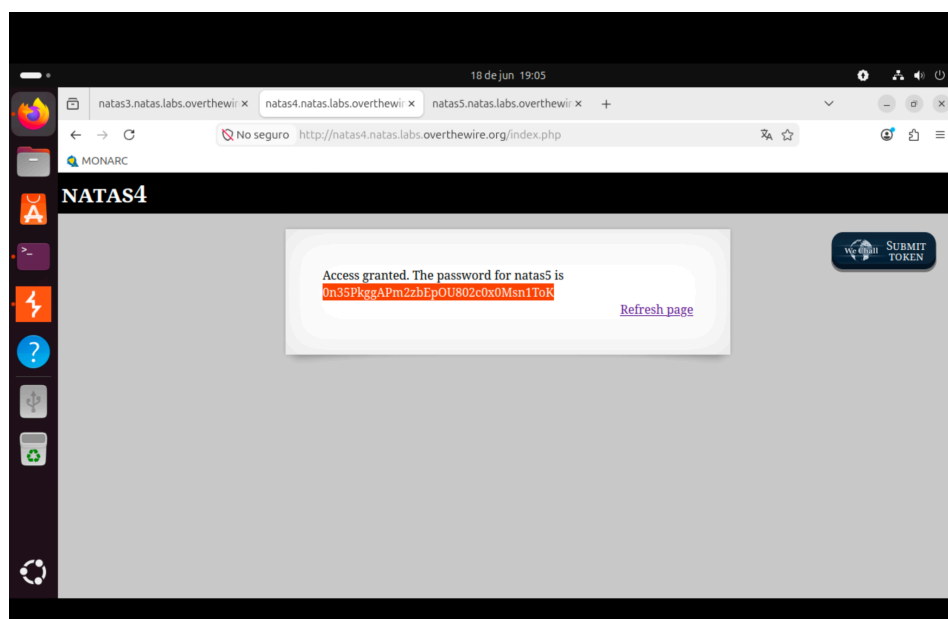
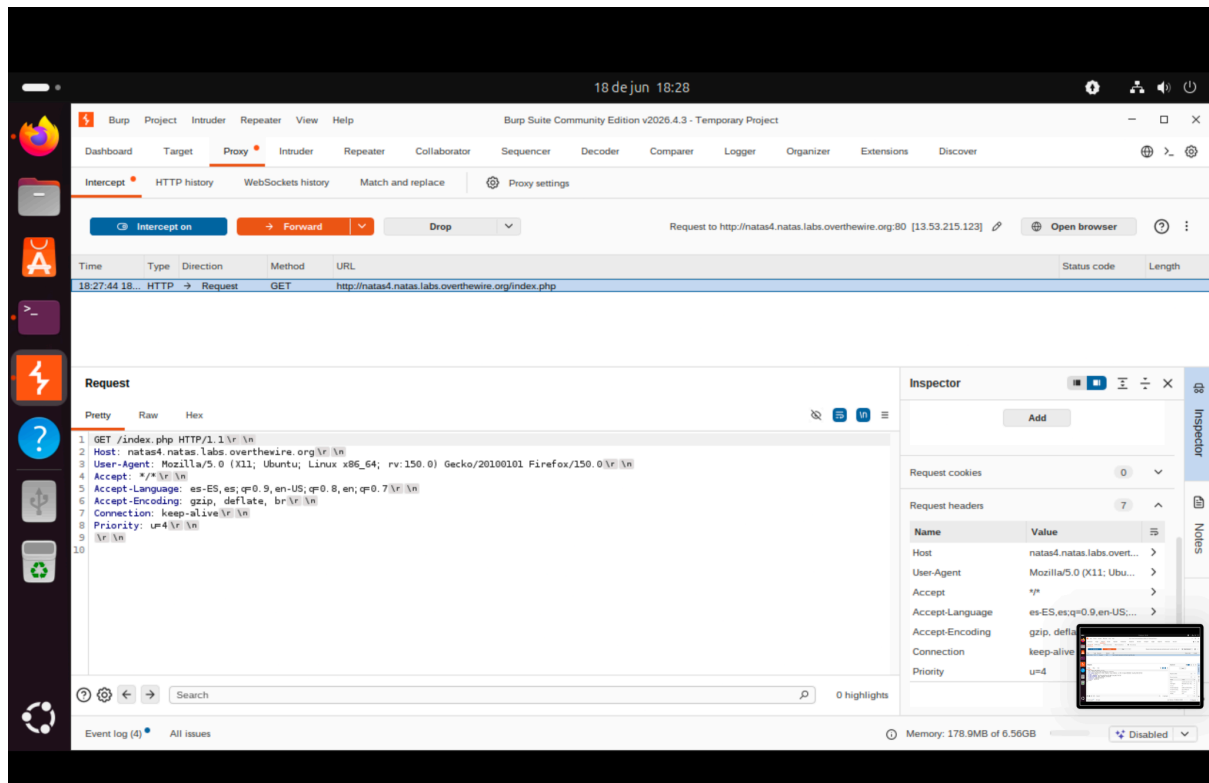
Evidencia:

La captura de pantalla muestra la directiva `Disallow` apuntando al directorio `/s3cr3t/`.

Mitigación (posible):

No utilizar el fichero robots.txt para ocultar recursos sensibles. Los directorios que contengan información restringida deben protegerse mediante controles de acceso adecuados, autenticación y autorización en el servidor.

NATAS5:



Imaginemos que Natas4 es una discoteca (por ejemplo, por decir algo)

- El portero nos dice:

«Solo podéis entrar si venís de Natas5».

Nosotras conocemos la contraseña de Natas4, pero eso no es suficiente. Además de comprobar quiénes somos, el portero quiere saber de dónde venimos.

Normalmente, nuestro navegador le dice la verdad:

Vengo de: ninguna parte

- Y el portero responde:

No podéis entrar.

- Lo que hacemos con Burp Suite es interceptar la conversación entre el navegador y el servidor antes de que llegue al portero. Burp nos permite modificar un dato llamado Referer, que indica supuestamente desde qué página venimos. Lo cambiamos por:

Referer: http://natas5.natas.labs.overthewire.org/

- Es como si le dijéramos al portero:

«Decid que venimos de Natas5».

- Entonces el portero piensa:

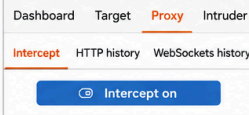
Perfecto, venís de Natas5. Aquí tenéis la contraseña del siguiente nivel.

NATAS4 → NATAS5

PASOS CON BURP SUITE

1 Activar interceptación

En Burp Suite, ve a la pestaña **Proxy** > **Intercept** y asegúrate de que el botón esté en "Intercept on" (azul).



2 Entrar en Natas4

Abre tu navegador e inicia sesión en:

<http://natas4.natas.labs.overthewire.org/>

Burp interceptará la petición y la mostrará.



3 Capturar la petición

Burp detiene la petición. La verás en la pestaña **Intercept**.

Time	Method	URL
12:34:56	GET	/



Selecciona la petición.

4 Ir a la vista RAW y añadir Referer

Cambia la vista a **RAW** y añade la cabecera **Referer** justo debajo de **Host**.

```

Raw
GET / HTTP/1.1
Host: natas4.natas.labs.overthewire.org
Referer: http://natas5.natas.labs.overthewire.org/
User-Agent: Mozilla/5.0
Accept: text/html,application/xhtml+xml,...
Accept-Language: es-ES, es;q=0.9,en-US;q=0.8
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
...
  
```

5 Forward (reenviar)

Pulsa el botón **Forward** para enviar la petición modificada al servidor.



Burp enviará la petición con el **Referer** modificado.

6 Obtener la respuesta

Burp mostrará la respuesta del servidor en la parte inferior.

```

Response
<html>
<body>
<h1>Access granted!</h1>
<p>Here is your password for natas5:</p>
<pre>natas5: ABCDEFGHIJKLMNOP</pre>
</body>
</html>
  
```

✓ Copia la contraseña de Natas5.

7 Acceder a Natas5

Usa la contraseña obtenida para iniciar sesión en:

<http://natas5.natas.labs.overthewire.org/>

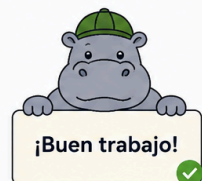
NATAS5

Username:

Password:

8 ¡Listo!

Ya estás dentro de **Natas5**. A por el siguiente reto. 🐻



IDEA CLAVE:

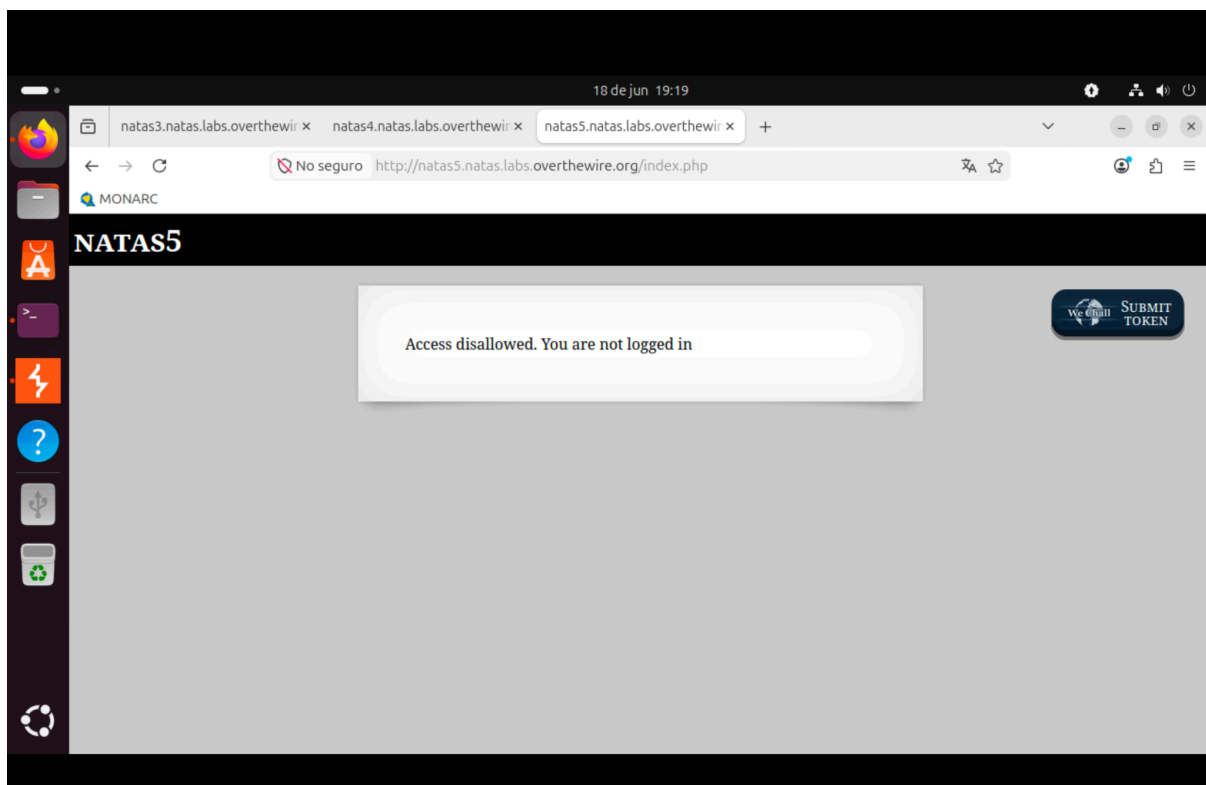
El servidor de Natas4 solo te da la contraseña si pareces venir de Natas5. La cabecera **Referer** puede falsificarse fácilmente, por eso no debe usarse como mecanismo de seguridad.

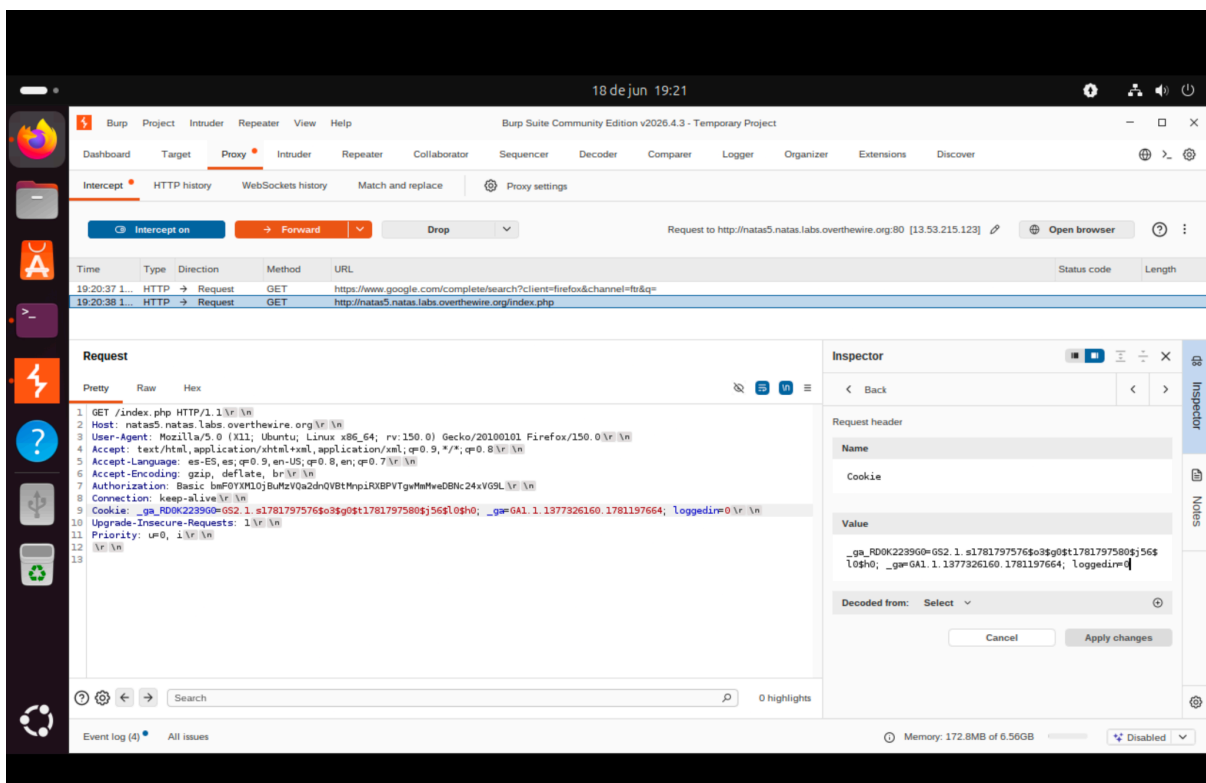
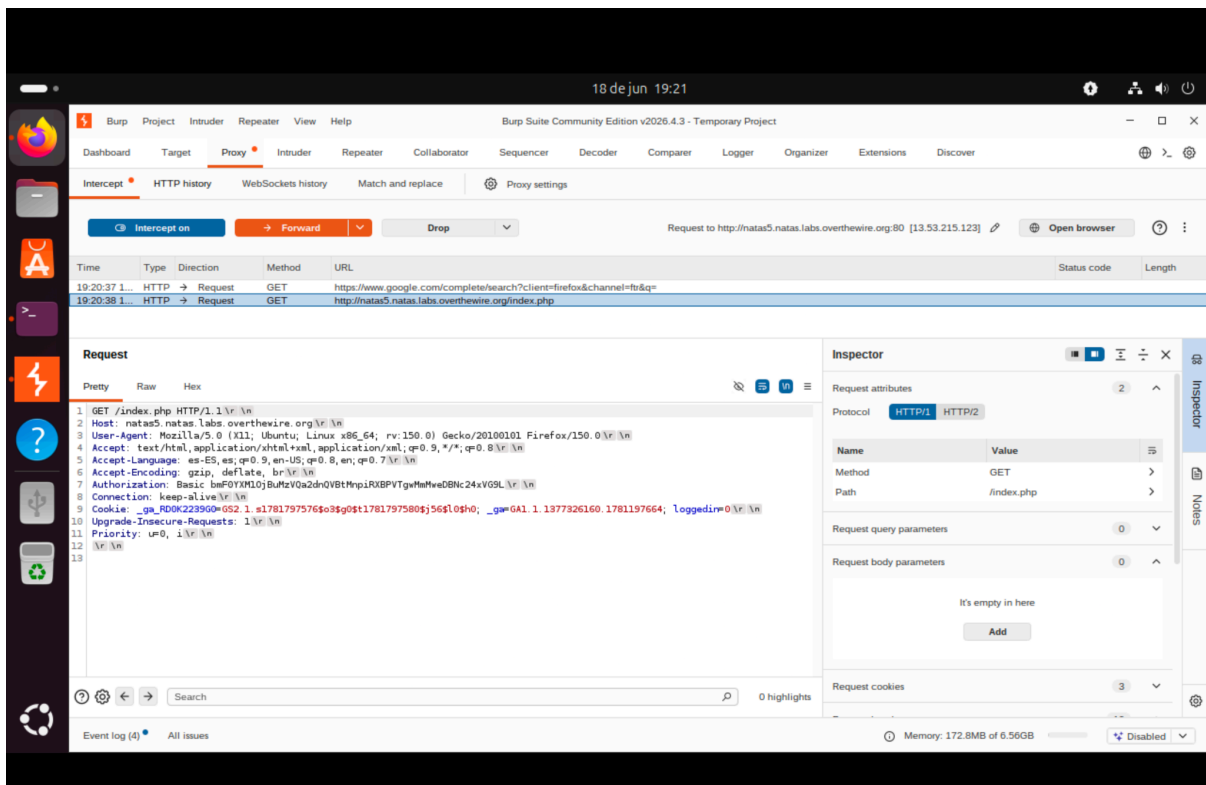
NAVEGADOR

BURP
(cambia Referer)

NATAS4
(te da la contraseña)

La lección de este reto es que la cabecera Referer puede falsificarse fácilmente, por lo que nunca debe utilizarse como mecanismo de seguridad.

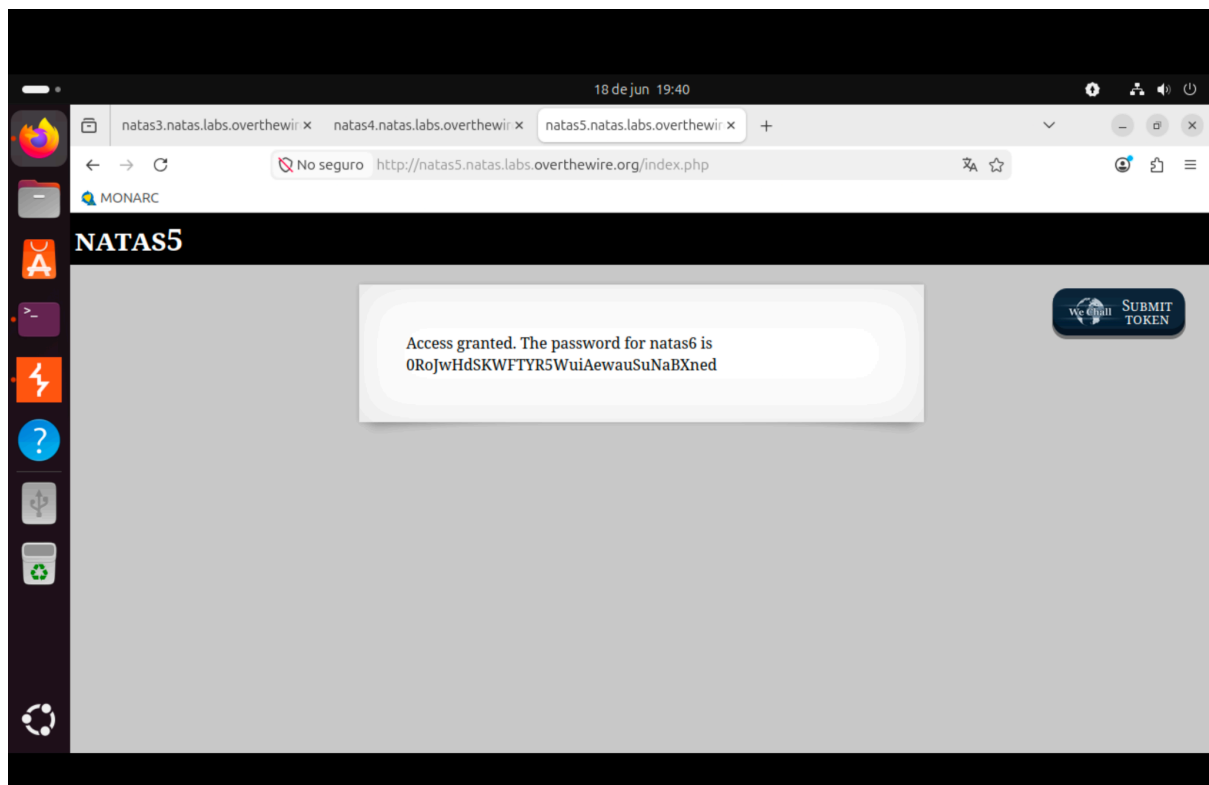


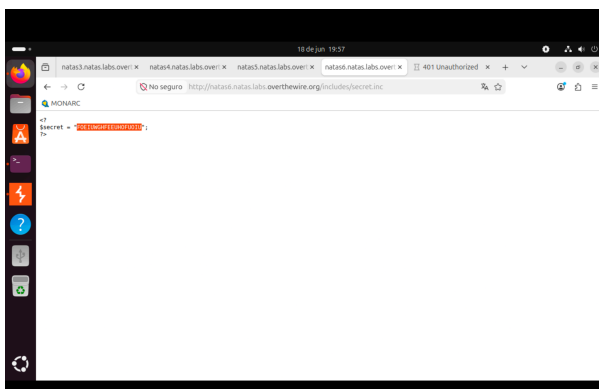
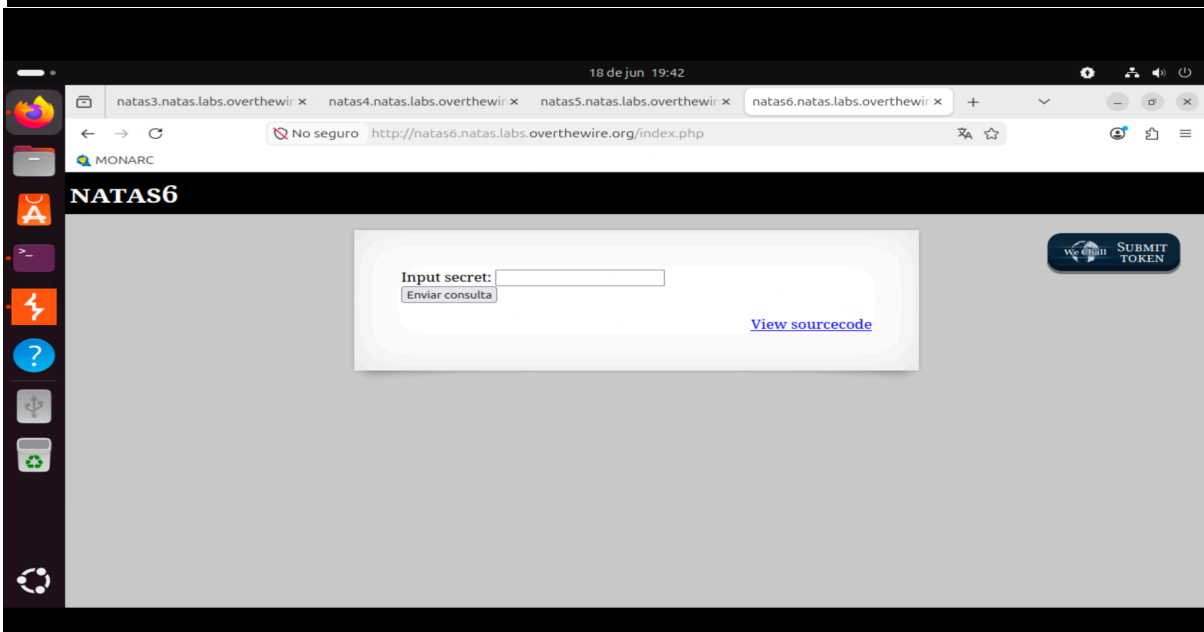
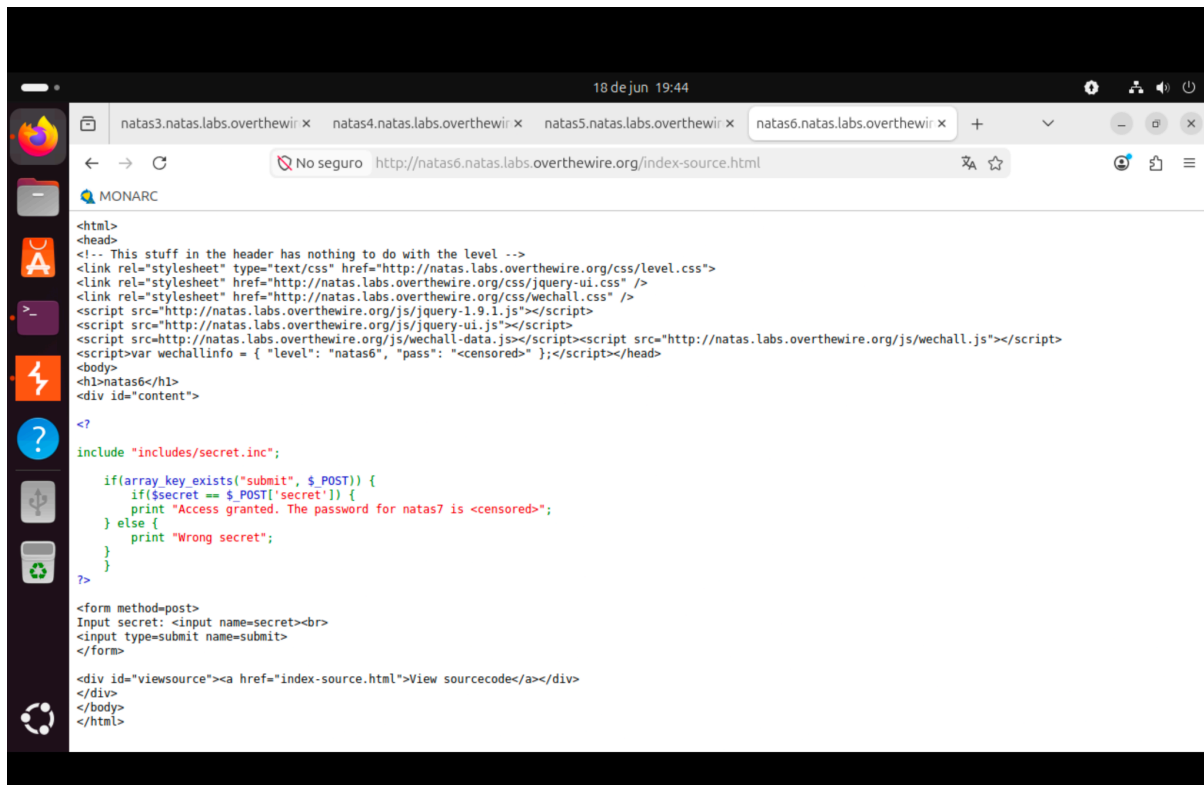


Al acceder a Natas5, la aplicación muestra el mensaje «Access disallowed. You are not logged in». Dado que no existe ningún formulario de autenticación visible, una

posible teoría, es que el estado de inicio de sesión se gestione mediante una **cookie**. Por lo tanto, se intercepta la petición con Burp Suite para analizar las cabeceras HTTP y comprobar si existe alguna cookie relacionada con la autenticación, verificando posteriormente si la aplicación confía en valores controlados por el cliente. Cambiamos el valor de la cookie de 0 a 1 eso da a entender que estamos autenticados...

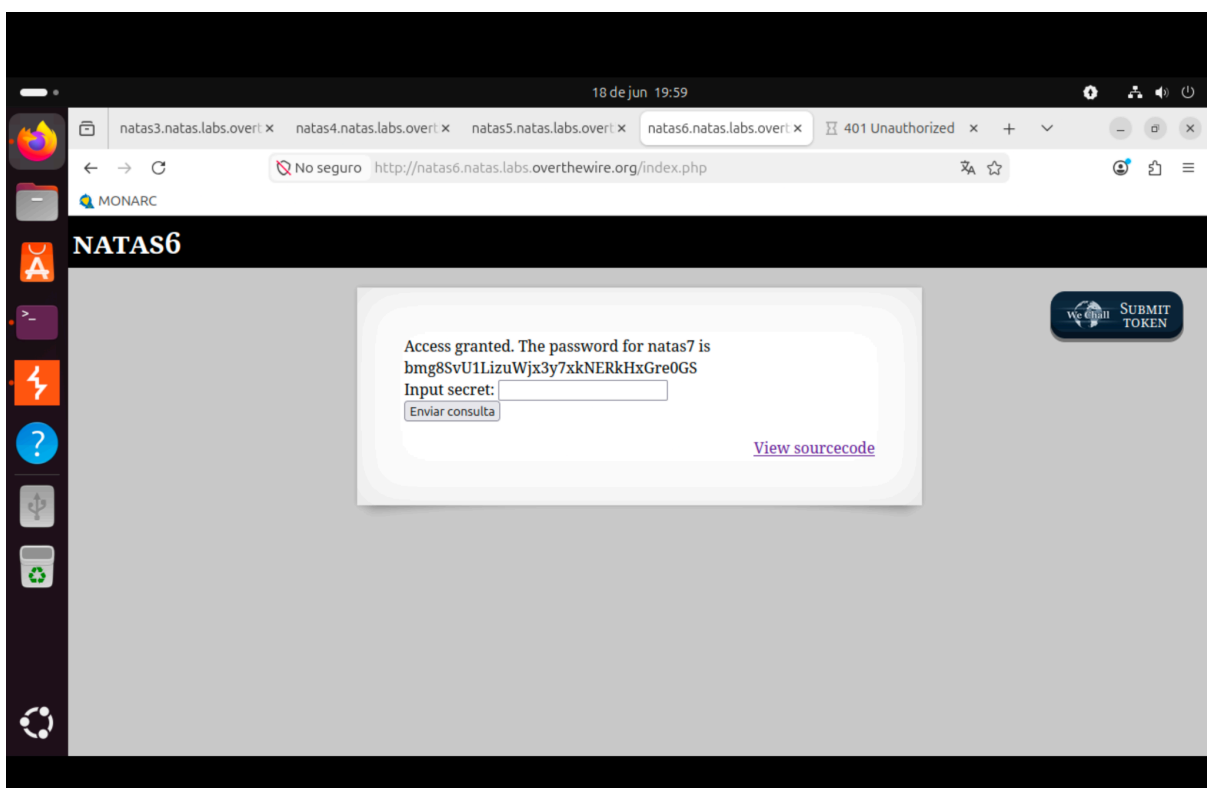
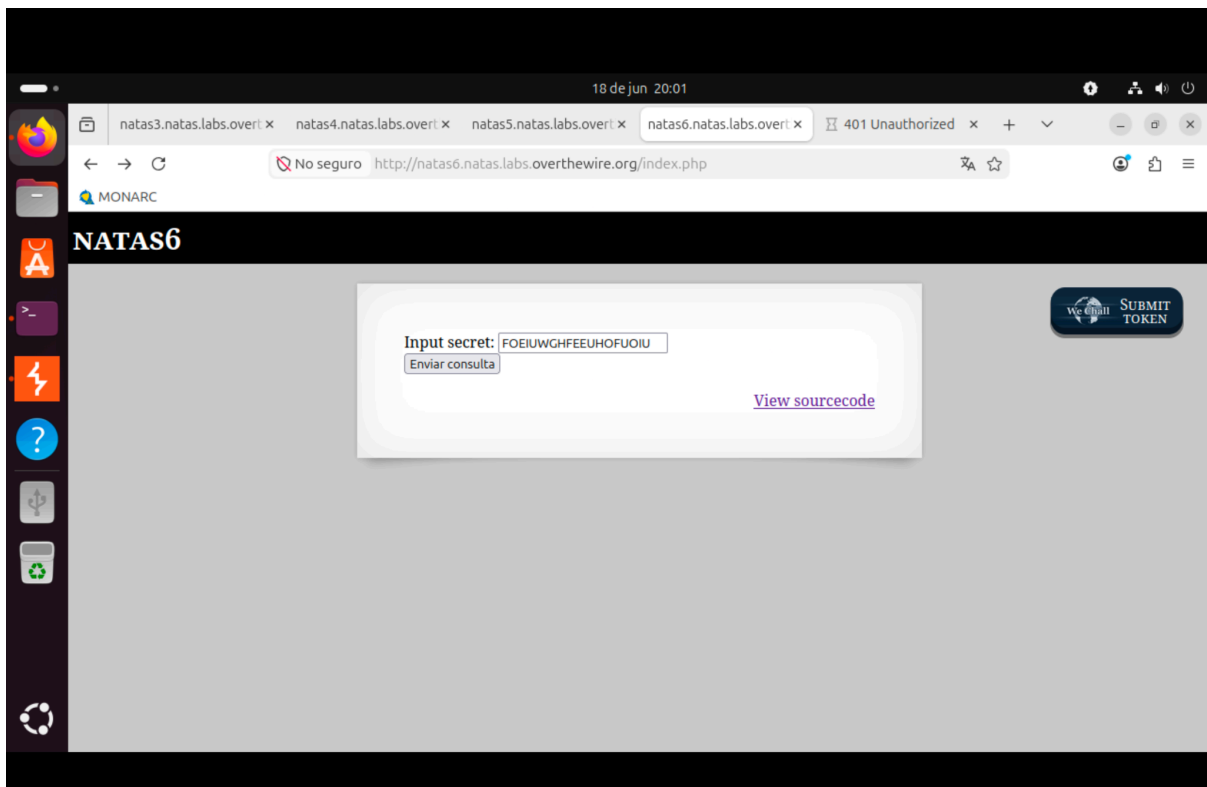
Forwardemos y

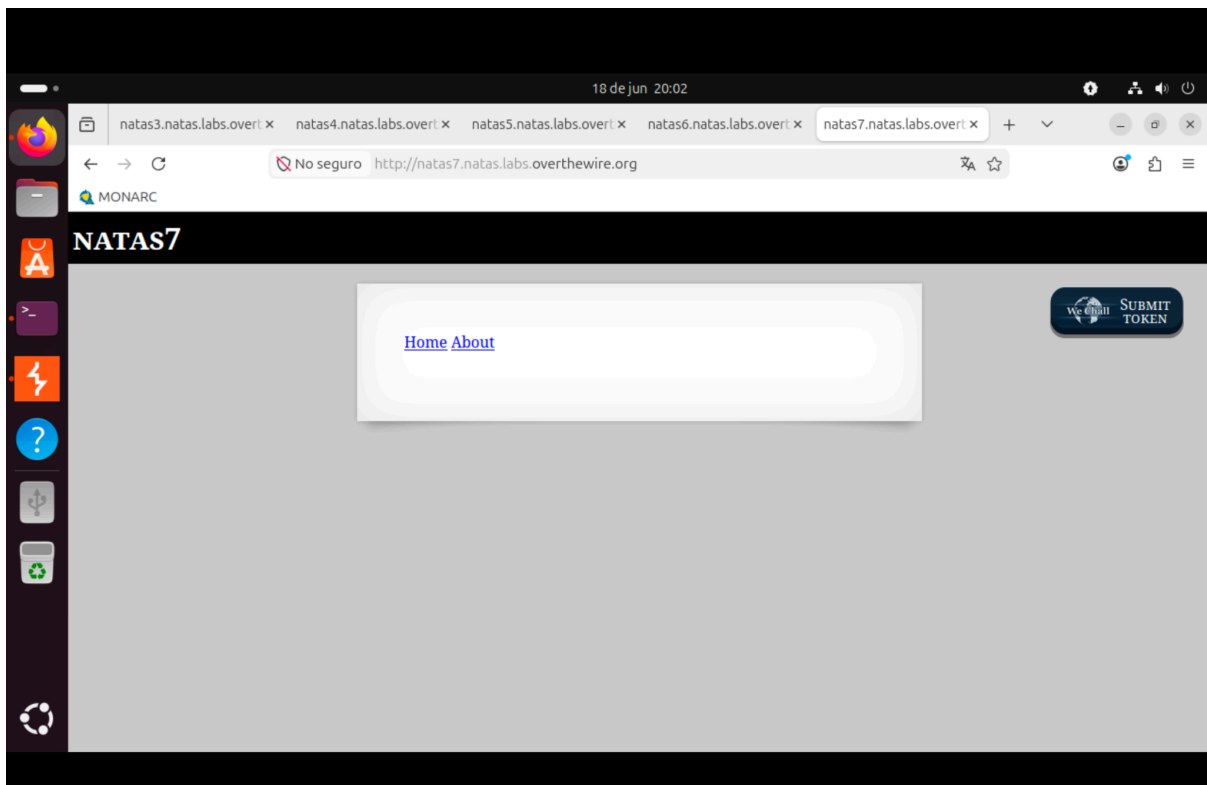




Hemos hecho caso a la pista de ver código fuente y vemos la pista... pero la contraseña no funciona...(ensored). Pero en el código hay otra pista en el include. Hay un

endpoint que quizás nos dé alguna pista por el nombre que tiene.





Vamos a pensar...

“Si la aplicación utiliza el valor del parámetro page para decidir qué archivo mostrar, ¿qué ocurre si introduzco la ruta de otro archivo en lugar de home o about?.”

Esto apunta a una posible vulnerabilidad de **Local File Inclusion (LFI)**¹.

*“El parámetro **page** parece indicar qué archivo carga el servidor. Voy a probar si puedo cambiarlo por otros archivos conocidos del sistema.”*

¹Una **Local File Inclusion (LFI)** es una vulnerabilidad que ocurre cuando una página web permite al usuario indicar qué archivo debe abrir el servidor sin comprobar si tiene permiso para acceder a él.

Por ejemplo, si una web utiliza una URL como:

`index.php?page=home`

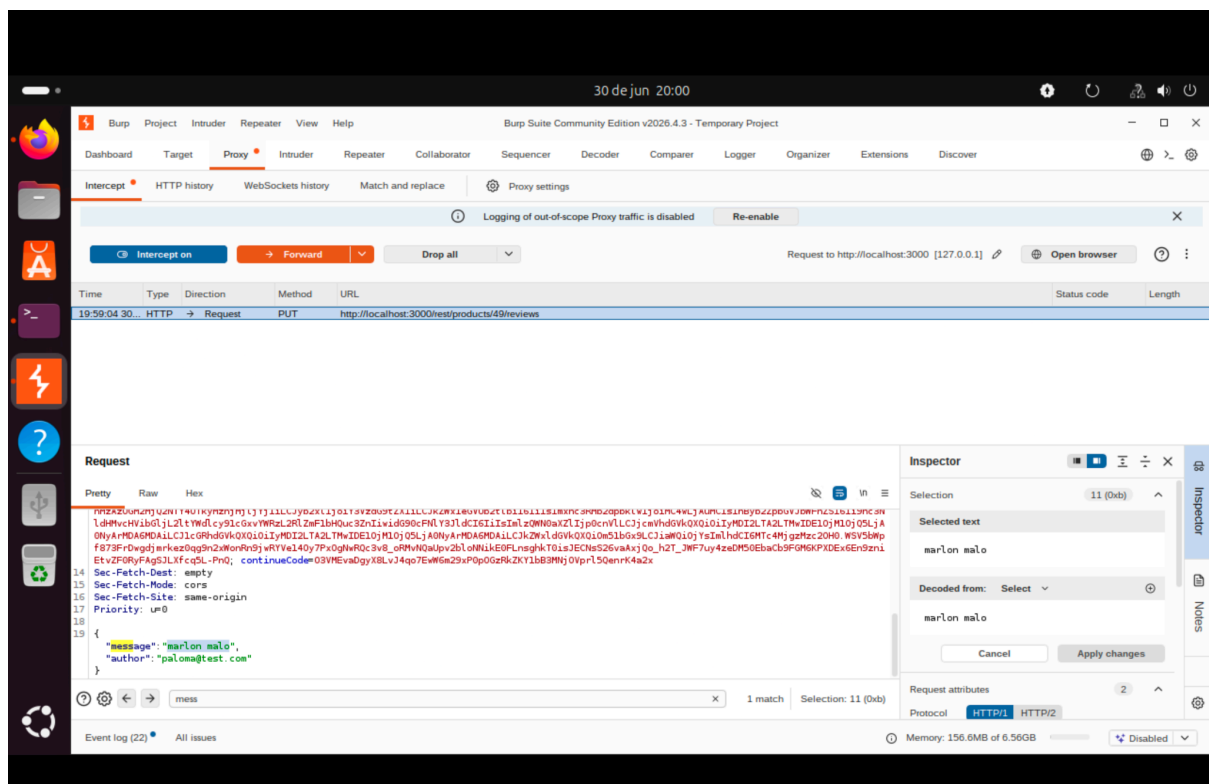
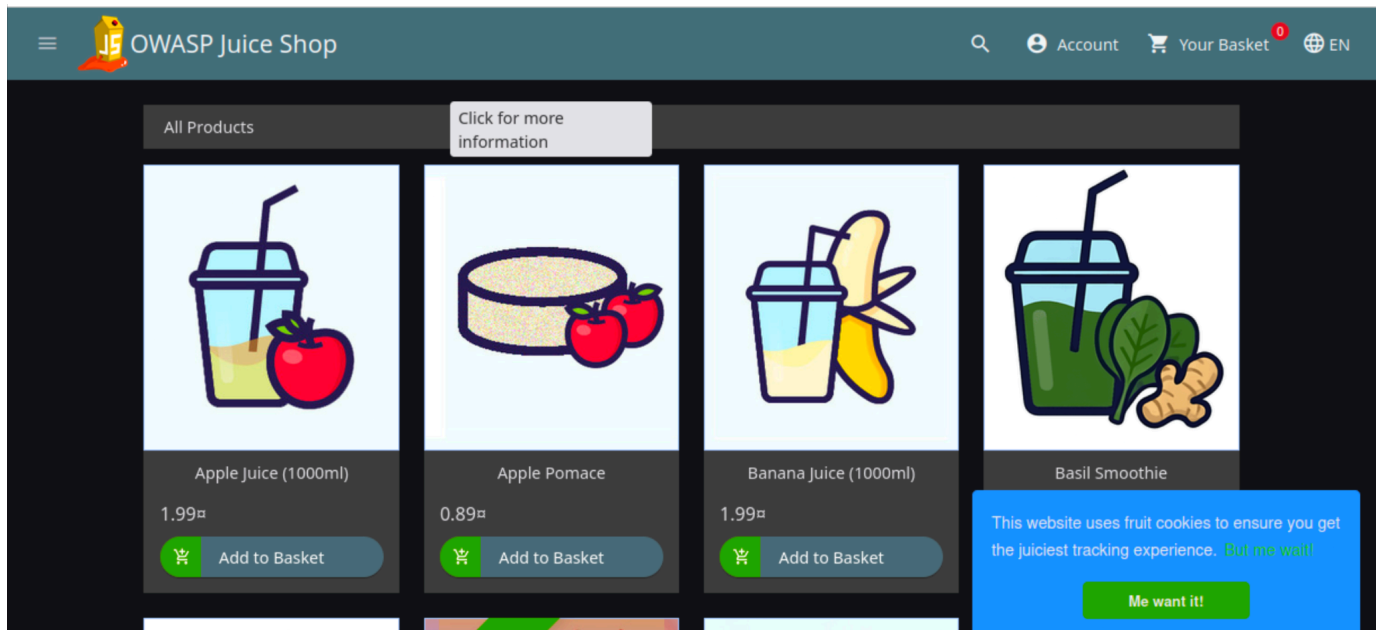
el servidor debería mostrar únicamente las páginas permitidas, como `home` o `about`. Sin embargo, si la aplicación no valida correctamente ese parámetro, un atacante puede sustituirlo por la ruta de un archivo interno del sistema, como:

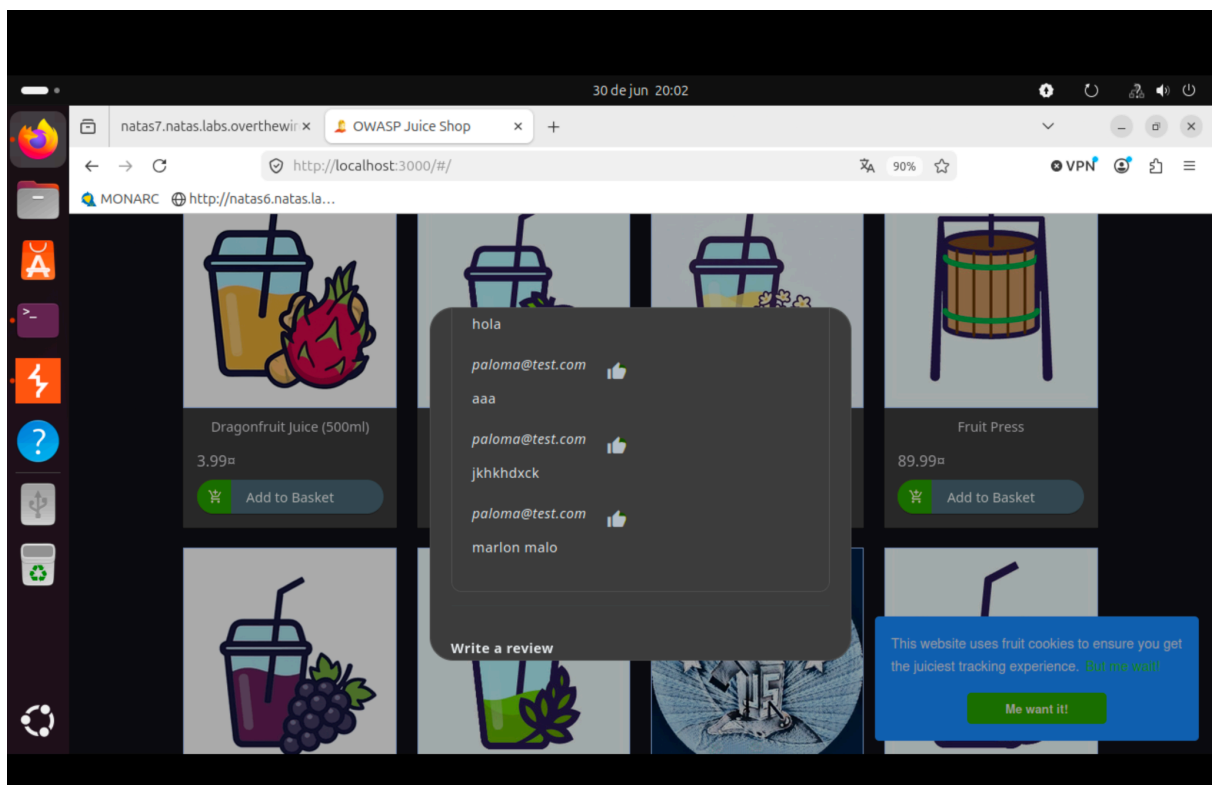
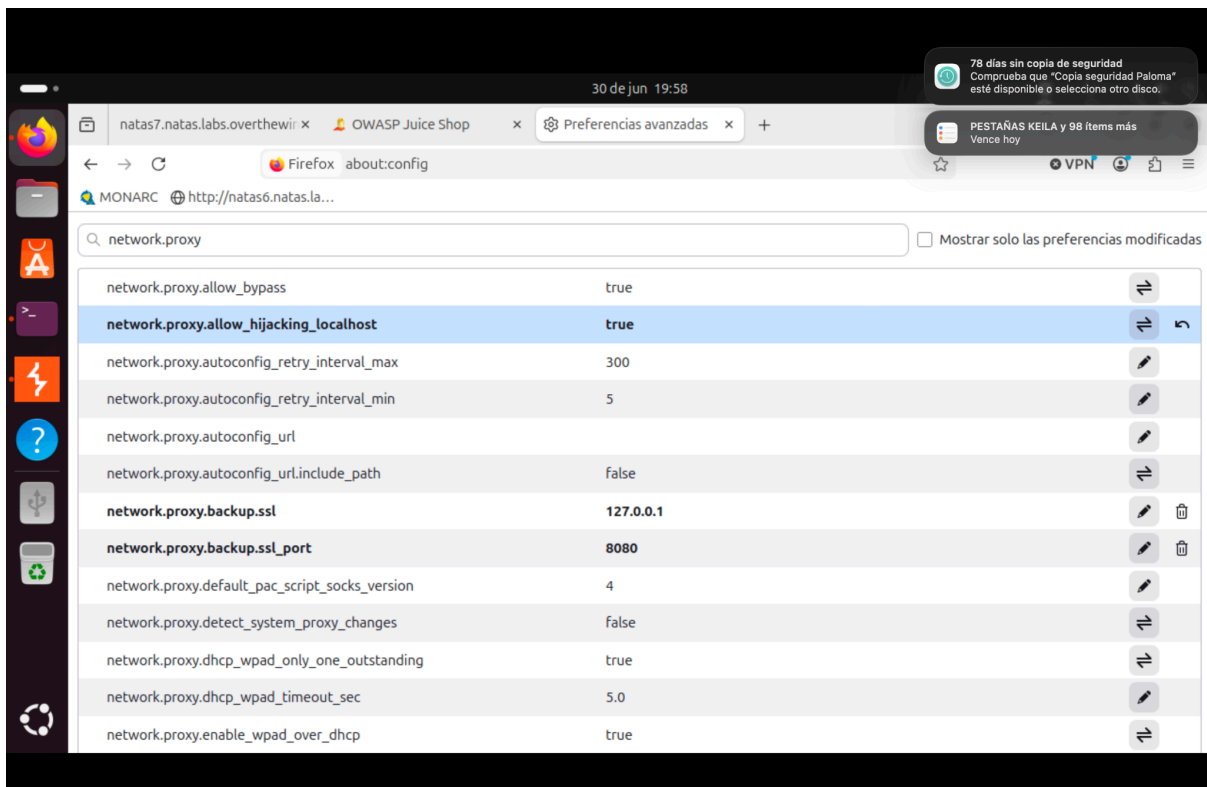
`index.php?page=/etc/natas_webpass/natas8`

De esta forma, el servidor podría mostrar información sensible, como contraseñas, archivos de configuración o datos internos. En pocas palabras:

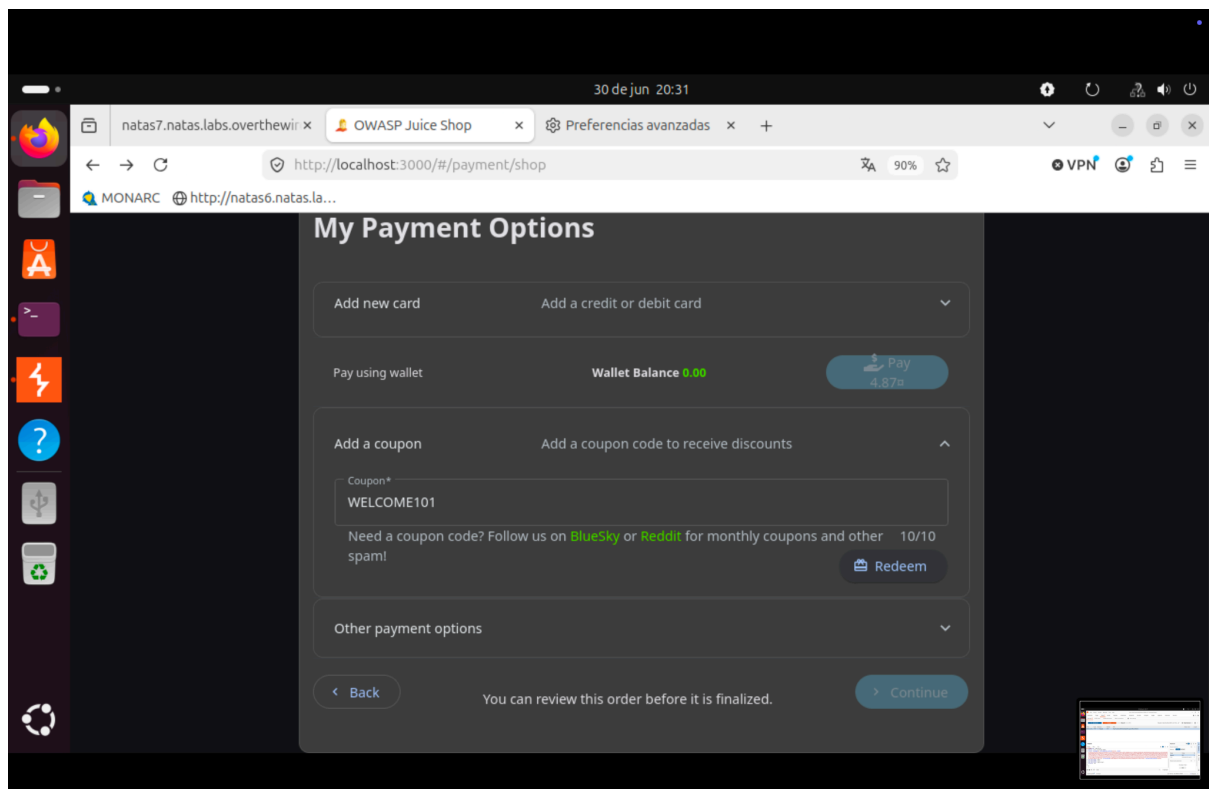
Una LFI permite al usuario decidir qué archivo lee el servidor, cuando esa decisión debería estar controlada únicamente por la aplicación.

JUICE-SHOP





Expired Coupon



Objetivo

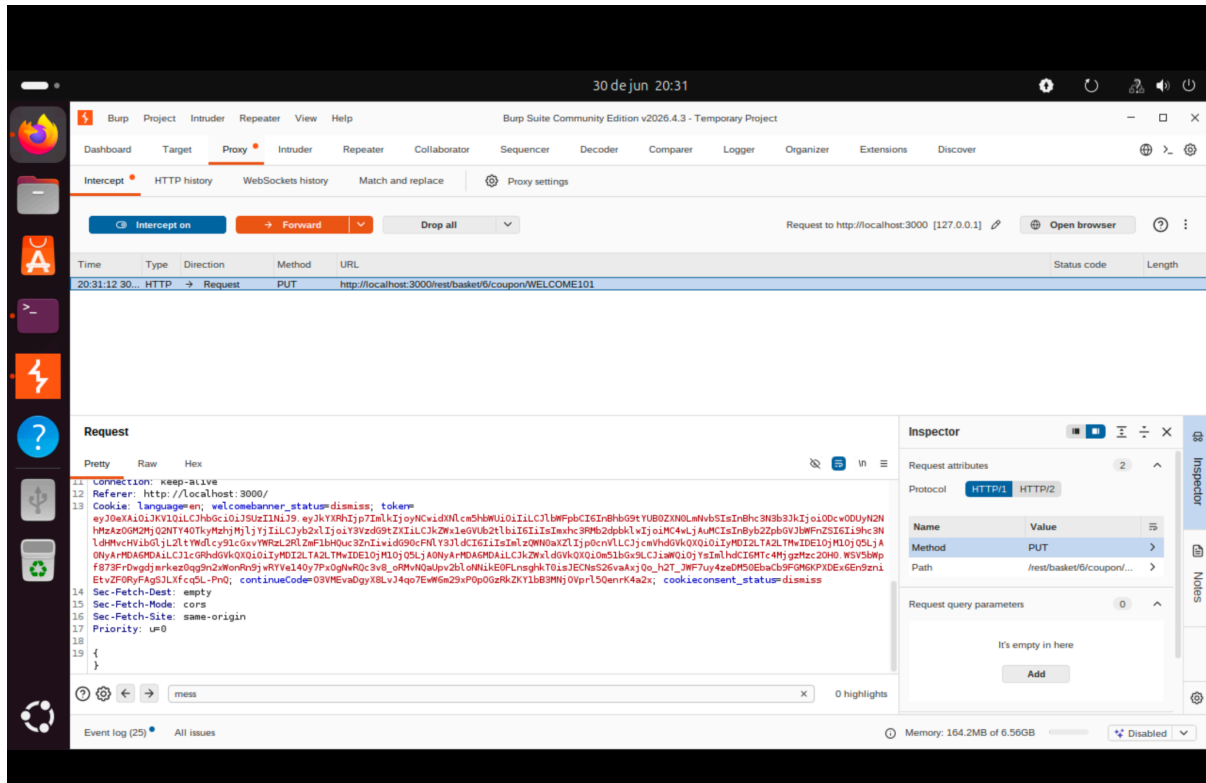
El objetivo de este reto consiste en analizar el funcionamiento del sistema de cupones de OWASP Juice Shop mediante la interceptación y modificación de las peticiones HTTP con Burp Suite, comprobando si es posible utilizar un cupón que no debería ser válido.

Procedimiento

1. Se añadió un producto al carrito de la compra.
2. Desde Burp Suite se interceptó la petición enviada al aplicar un cupón de descuento.
3. La petición fue enviada a la herramienta Repeater para poder modificarla y reenviarla manualmente.
4. Se modificó el código del cupón en la URL de la petición HTTP para comprobar la respuesta del servidor.

Evidencia

Figura X. Petición HTTP interceptada y enviada a Burp Repeater.



En la captura puede observarse la petición:

PUT /rest/basket/6/coupon/WELCOME101 HTTP/1.1

Host: localhost:3000

Tras reenviar la petición, el servidor respondió con:

HTTP/1.1 404 Not Found

Invalid coupon

Lo anterior indica que el cupón introducido no existe o no es reconocido por la aplicación, por lo que el servidor rechaza la solicitud y devuelve un código de estado 404 junto con el mensaje “Invalid coupon”.

Conclusión

Burp Suite permitió interceptar y modificar manualmente la petición HTTP enviada por el cliente. La respuesta obtenida demuestra que el servidor valida la existencia del cupón antes de aplicarlo. Para completar el reto Expired Coupon sería necesario identificar un cupón existente pero caducado y modificar la petición correspondiente para analizar el comportamiento de la aplicación frente a dicho escenario.

Puedes redactarlo así, con un tono técnico propio de un informe de auditoría:

Intento de manipulación de la respuesta del servidor

Durante el análisis se planteó la posibilidad de modificar manualmente la respuesta devuelta por el servidor, sustituyendo el mensaje Invalid coupon por un mensaje que indicara que el cupón era válido. Sin embargo, esta técnica no produjo ningún efecto sobre el funcionamiento de la aplicación.

La razón es que el mensaje Invalid coupon forma parte de la respuesta HTTP generada por el servidor una vez este ha procesado la solicitud y determinado que el cupón introducido no es válido. Modificar dicho texto únicamente altera la visualización local de la respuesta en Burp Suite, pero no modifica el estado interno de la aplicación ni obliga al servidor a aceptar el cupón.

Este comportamiento demuestra que la validación de los cupones se realiza en el lado del servidor (server-side validation). En consecuencia, para superar este reto sería necesario modificar la petición enviada al servidor o utilizar un cupón existente que cumpla las condiciones del desafío, ya que alterar únicamente la respuesta recibida no influye en la lógica de validación implementada por la aplicación.

Este texto demuestra que entiendes la diferencia entre manipular una petición y manipular una respuesta, un concepto muy importante en una auditoría de seguridad.

